

What are Semantic Kernel Vector Stores? (Preview)



Tip

If you are looking for information about the legacy Memory Store connectors, refer to the [Memory Stores page](#).

Vector databases have many use cases across different domains and applications that involve natural language processing (NLP), computer vision (CV), recommendation systems (RS), and other areas that require semantic understanding and matching of data.

One use case for storing information in a vector database is to enable large language models (LLMs) to generate more relevant and coherent responses. Large language models often face challenges such as generating inaccurate or irrelevant information; lacking factual consistency or common sense; repeating or contradicting themselves; being biased or offensive. To help overcome these challenges, you can use a vector database to store information about different topics, keywords, facts, opinions, and/or sources related to your desired domain or genre. The vector database allows you to efficiently find the subset of information related to a specific question or topic. You can then pass information from the vector database with your prompt to your large language model to generate more accurate and relevant content.

For example, if you want to write a blog post about the latest trends in AI, you can use a vector database to store the latest information about that topic and pass the information along with the ask to a LLM in order to generate a blog post that leverages the latest information.

For python, Semantic Kernel provides an abstraction for interacting with Vector Stores and a list of out-of-the-box implementations that implement these abstractions for various databases.

For .net an abstraction is provided by the .net Foundation. Various implementations that implement these abstractions are shipped by database providers and the .net Foundation via the [community toolkit](#) .

Features include creating, listing and deleting collections of records, and uploading, retrieving and deleting records. The abstraction makes it easy to experiment with a free or locally hosted Vector Store and then switch to a service when needing to scale up.

Retrieval Augmented Generation (RAG) with Vector Stores

The vector store abstraction is a low level api for adding and retrieving data from vector stores. Semantic Kernel has built-in support for using any one of the Vector Store implementations for RAG. This is achieved by wrapping `IVectorSearchable<TRecord>` and exposing it as a Text Search implementation.

💡 Tip

To learn more about how to use vector stores for RAG see [How to use Vector Stores with Semantic Kernel Text Search](#).

💡 Tip

To learn more about text search see [What is Semantic Kernel Text Search?](#)

💡 Tip

To learn more about how to quickly add RAG into your agent see [Adding Retrieval Augmented Generation \(RAG\) to Semantic Kernel Agents](#).

The Vector Store Abstraction

The Vector Store abstractions are provided in the [Microsoft.Extensions.VectorData.Abstractions](#) nuget package. The following are the main abstract base classes and interfaces.

Microsoft.Extensions.VectorData.VectorStore

`VectorStore` contains operations that spans across all collections in the vector store, e.g. `ListCollectionNames`. It also provides the ability to get `VectorStoreCollection<TKey, TRecord>` instances.

Microsoft.Extensions.VectorData.VectorStoreCollection<TKey, TRecord>

`VectorStoreCollection<TKey, TRecord>` represents a collection. This collection may or may not exist, and the abstract base class provides methods to check if the collection exists, create it or delete it. The abstract base class also provides methods to upsert, get and delete records. Finally, the abstract base class inherits from `IVectorSearchable<TRecord>` providing vector search capabilities.

Microsoft.Extensions.VectorData.IVectorSearchable<TRecord>

- `SearchAsync<TRecord>` can be used to do either:
 - vector searches taking some input that can be vectorized by a registered embedding generator or by the vector database where the database supports this.
 - vector searches taking a vector as input.

Getting started with Vector Stores

Import the necessary nuget packages

All the vector store interfaces and any abstraction related classes are available in the `Microsoft.Extensions.VectorData.Abstractions` nuget package. Each vector store implementation is available in its own nuget package. For a list of known implementations, see the [Out-of-the-box connectors page](#).

The abstractions package can be added like this.

```
.NET CLI
```

```
dotnet add package Microsoft.Extensions.VectorData.Abstractions
```

Define your data model

The Vector Store abstractions use a model first approach to interacting with databases. This means that the first step is to define a data model that maps to the storage schema. To help the implementations create collections of records and map to the storage schema, the model can be annotated to indicate the function of each property.

```
C#
```

```
using Microsoft.Extensions.VectorData;  
  
public class Hotel  
{
```

```
[VectorStoreKey]
public ulong HotelId { get; set; }

[VectorStoreData(IsIndexed = true)]
public string HotelName { get; set; }

[VectorStoreData(IsFullTextIndexed = true)]
public string Description { get; set; }

[VectorStoreVector(Dimensions: 4, DistanceFunction =
DistanceFunction.CosineSimilarity, IndexKind = IndexKind.Hnsw)]
public ReadOnlyMemory<float>? DescriptionEmbedding { get; set; }

[VectorStoreData(IsIndexed = true)]
public string[] Tags { get; set; }
}
```

💡 Tip

For more information on how to annotate your data model, refer to [defining your data model](#).

💡 Tip

For an alternative to annotating your data model, refer to [defining your schema with a record definition](#).

Connect to your database and select a collection

Once you have defined your data model, the next step is to create a VectorStore instance for the database of your choice and select a collection of records.

In this example, we'll use Qdrant. You will therefore need to import the Qdrant nuget package.

.NET CLI

```
dotnet add package CommunityToolkit.VectorData.Qdrant
```

If you want to run Qdrant locally using Docker, use the following command to start the Qdrant container with the settings used in this example.

```
cli
```

```
docker run -d --name qdrant -p 6333:6333 -p 6334:6334 qdrant/qdrant:latest
```

To verify that your Qdrant instance is up and running correctly, visit the Qdrant dashboard that is built into the Qdrant docker container: <http://localhost:6333/dashboard>

Since databases support many different types of keys and records, we allow you to specify the type of the key and record for your collection using generics. In our case, the type of record will be the `Hotel` class we already defined, and the type of key will be `ulong`, since the `HotelId` property is a `ulong` and Qdrant only supports `Guid` or `ulong` keys.

C#

```
using CommunityToolkit.VectorData.Qdrant;
using Qdrant.Client;

// Create a Qdrant VectorStore object
var vectorStore = new QdrantVectorStore(new QdrantClient("localhost"), ownsClient:
true);

// Choose a collection from the database and specify the type of key and record
stored in it via Generic parameters.
var collection = vectorStore.GetCollection<ulong, Hotel>("skhotels");
```

Tip

For more information on what key and field types each Vector Store implementation supports, refer to [the documentation for each implementation](#).

Create the collection and add records

C#

```
// Placeholder embedding generation method.
async Task<ReadOnlyMemory<float>> GenerateEmbeddingAsync(string textToVectorize)
{
    // your logic here
}

// Create the collection if it doesn't exist yet.
await collection.EnsureCollectionExistsAsync();

// Upsert a record.
string descriptionText = "A place where everyone can be happy.";
ulong hotelId = 1;
```

```
// Create a record and generate a vector for the description using your chosen embedding generation implementation.
await collection.UpsertAsync(new Hotel
{
    HotelId = hotelId,
    HotelName = "Hotel Happy",
    Description = descriptionText,
    DescriptionEmbedding = await GenerateEmbeddingAsync(descriptionText),
    Tags = new[] { "luxury", "pool" }
});

// Retrieve the upserted record.
Hotel? retrievedHotel = await collection.GetAsync(hotelId);
```

Tip

For more information on how to generate embeddings see [embedding generation](#).

Do a vector search

C#

```
// Placeholder embedding generation method.
async Task<ReadOnlyMemory<float>> GenerateEmbeddingAsync(string textToVectorize)
{
    // your logic here
}

// Generate a vector for your search text, using your chosen embedding generation implementation.
ReadOnlyMemory<float> searchVector = await GenerateEmbeddingAsync("I'm looking for a hotel where customer happiness is the priority.");

// Do the search.
var searchResult = collection.SearchAsync(searchVector, top: 1);

// Inspect the returned hotel.
await foreach (var record in searchResult)
{
    Console.WriteLine("Found hotel description: " + record.Record.Description);
    Console.WriteLine("Found record score: " + record.Score);
}
```

Tip

For more information on how to generate embeddings see [embedding generation](#).

Next steps

[Learn about the Vector Store data architecture](#)[How to ingest data into a Vector Store](#)

Last updated on 07/01/2025